CAPSTONE PROJECT REPORT

AI / ML INTERNSHIP PROGRAM

PROJECT TITLE

CareerCompass AI — AI-Powered Resume Analyzer and Career Recommendation Platform

NAME	Souvik Roy
INTERNSHIP NAME	IBM-PBEL AI Internship for Experiential Learning
PROJECT LINK	https://github.com/Souvik313/Career_Guide-AI
LIVE DEMO	https://career-guide-ai-mu.vercel.app/
DATE OF SUBMISSION	July 2026

Abstract

Resume screening today is still, in a lot of places, done the old way — a recruiter skims a document for a few seconds and moves on, or a keyword-matching Applicant Tracking System filters candidates out based on exact word overlap. Both approaches miss something important: two people can describe the same skill in completely different words, and a purely lexical system has no way to know that. This project, CareerCompass AI, was built to explore whether a semantic approach — one that understands meaning rather than just matching strings — could do better. The platform takes a candidate's resume (PDF or DOCX), extracts and cleans the text, generates dense vector embeddings using Sentence Transformers, and compares those embeddings against a dataset of roughly 142,000 real job descriptions using cosine similarity. On top of this matching layer sits a Groq-hosted LLM (Llama 3.3 70B) that turns the raw numbers into a readable, personalized career report — covering strengths, missing skills, and a rough learning roadmap. The whole thing is wrapped in a React frontend with an analytics dashboard and one-click PDF export. This report documents the motivation, design, implementation, and honest limitations of the system as it stands today.

Table of Contents

1. Introduction

Every year, a huge number of resumes get filtered out not because the candidate lacks the right skills, but because they described those skills differently from how the job posting phrased them. A resume that says "built REST APIs" and a job description that asks for "backend service development" mean roughly the same thing to a human reader, but a plain keyword-matching system will often treat them as unrelated. This mismatch is one of the more frustrating, and honestly avoidable, problems in the hiring pipeline today.

CareerCompass AI started as an attempt to fix a smaller, more personal version of this problem: helping a candidate understand, in plain language, where their resume stands relative to the roles they're aiming for — not just "here are five jobs," but "here is your match score, here is what you're missing, and here is a reasonable path to close that gap." Along the way it grew into a fairly complete full-stack application that combines resume parsing, semantic search, and LLM-based report generation, backed by a React dashboard and PDF export.

This report walks through why the project exists, how it is structured, the reasoning behind the technical choices, and what still needs work — the last part being just as important as the rest, since being honest about a project's limitations is, in my view, part of doing the engineering properly.

2. Problem Statement

Traditional resume screening — whether done manually by a recruiter or automatically by a keyword-based ATS — suffers from three recurring issues:

- It relies on exact or near-exact keyword overlap, which penalizes candidates who use different but semantically equivalent terminology.
- It provides little to no actionable feedback — a rejected candidate rarely learns what specific skills would have improved their chances.
- It does not scale well to give every candidate a fair, individualized read — recruiters spend seconds per resume, not minutes.

CareerCompass AI addresses this by replacing keyword overlap with semantic similarity, and by using an LLM to generate the kind of individualized, readable feedback a career counselor might give — at a scale a human reviewer simply cannot match.

3. Objectives

- Automate resume text extraction and skill identification from PDF and DOCX files.
- Replace keyword-based job matching with semantic similarity search using Sentence Transformers.
- Identify skill gaps between a candidate's resume and their target career paths.
- Generate a personalized, LLM-written career report covering strengths, gaps, and a learning roadmap.
- Present all of this through an interactive analytics dashboard rather than a static text block.
- Allow the candidate to export a professional, shareable PDF version of their report.
- Build the whole thing as a production-shaped full-stack application, not a notebook prototype.

4. Related Work and Motivation

Most commercial Applicant Tracking Systems (ATS) in use today are still fundamentally keyword matchers — they parse a resume, extract a bag of terms, and score it against a job description's own bag of terms. This is fast and cheap to run at scale, which is exactly why it's so widespread, but it is also the reason candidates are frequently told to "tailor their resume" by stuffing in exact phrases from the job posting. That advice exists because the system genuinely cannot tell that two phrasings mean the same thing.

Sentence embedding models change this picture. Instead of comparing raw tokens, a Sentence Transformer maps an entire sentence or paragraph into a dense vector such that semantically similar text ends up close together in that vector space, regardless of the exact words used. Comparing two pieces of text then becomes a matter of computing the cosine similarity between their vectors — a well-understood, fast operation. This is the same general idea behind modern semantic search engines, and it is the core technical bet this project is built around: that semantic similarity is a better proxy for "is this candidate a good fit" than raw keyword overlap.

On the reporting side, large language models like Llama 3.3 70B (accessed here through the Groq API for fast inference) make it possible to turn a handful of structured numbers — a match score, a list of missing skills — into a coherent, readable narrative, which is something no purely statistical pipeline could do on its own.

5. System Architecture

The system follows a fairly standard client-server split: a React frontend handles everything the user sees and interacts with, while a FastAPI backend owns resume parsing, embedding generation, semantic matching, and report generation. The backend in turn talks to two external dependencies — the local Sentence Transformer model for embeddings, and the Groq API for the LLM-generated career report.

High-level flow:

- The user uploads a resume (PDF or DOCX) through the React frontend.
- The frontend sends the file to the FastAPI backend over a REST endpoint.
- The backend extracts raw text using PyMuPDF (for PDFs) or python-docx (for DOCX files).
- NLP preprocessing cleans the text and extracts a candidate skill list.
- A Sentence Transformer model converts the cleaned resume text into a dense embedding vector.
- That embedding is compared against the pre-computed embeddings of ~142,000 job descriptions using cosine similarity, producing a ranked list of matches.
- A Career Advisor module derives an overall match score, missing skills, strengths, and candidate career paths from the matching results.
- This structured output is passed to the Groq-hosted Llama 3.3 70B model, which writes the final personalized career report.
- The frontend renders everything through the dashboard and analytics views, and offers a PDF export using jsPDF.

The design deliberately keeps these stages decoupled — the embedding model, the dataset, and the LLM layer can each be swapped or upgraded independently without touching the rest of the pipeline.

This separation of concerns was a conscious choice rather than something that happened by accident; it's the same reason the frontend and backend are two separate deployable services rather than a single monolith.

6. Dataset

Semantic job matching needs a reasonably large, real-world set of job descriptions to compare against — matching a resume against a handful of hand-written sample postings would not say much about how the system performs in practice. For this, the project uses the publicly available **lang-uk/recruitment-dataset-job-descriptions-english** dataset hosted on Hugging Face.

Detail	Description
Source	Hugging Face — lang-uk/recruitment-dataset-job-descriptions-english
Size	~142,000 job listing rows
Fields used	Job title, description, required skills
Refresh cadence	Static snapshot (not live-updated)
License	MIT (compatible with this project's own MIT license)

It's worth being upfront that this dataset is used purely for similarity matching at inference time — it is not used to train or fine-tune any model. The Sentence Transformer used for embeddings is a pre-trained general-purpose model, and the job description embeddings are pre-computed once and stored so that new resumes can be matched against them without recomputing the entire dataset each time.

7. Methodology and Implementation

7.1 Resume Parsing

Resumes arrive as either PDF or DOCX files. PDF text is extracted using PyMuPDF (fitz), which handles the layout-aware extraction reasonably well for most standard resume formats. DOCX files are parsed with python-docx. In both cases the goal at this stage is just to get clean, extractable plain text out of whatever formatting the original document used.

7.2 NLP Preprocessing and Skill Extraction

Once raw text is available, it goes through cleaning and normalization (removing boilerplate, fixing whitespace and encoding artifacts) and a skill-extraction step that pulls out technical skills the candidate has listed. This is the layer that turns an unstructured document into something the rest of the pipeline can actually reason about.

7.3 Semantic Embeddings and Matching

The cleaned resume text is converted into a dense vector embedding using a Sentence Transformer model. The same model is used offline to pre-compute embeddings for every job description in the dataset, so at request time the system only needs to embed the new resume and compare it against the already-computed job vectors. Similarity is measured using cosine similarity, and the top-matching roles are returned along with their similarity scores.

7.4 Career Analysis

From the ranked matches, the Career Advisor module computes an overall match score, identifies the candidate's strongest overlapping skills, flags the skills that are commonly required but missing from the resume, and shortlists the most relevant career paths.

7.5 AI-Generated Career Report

This structured output — scores, skill lists, candidate role suggestions — is passed as context to the Groq API, which runs inference on Llama 3.3 70B. The model turns these numbers into a written career summary, an explanation of the recommended career path, a strength analysis, a skill-gap analysis, and a short learning roadmap. Groq was chosen specifically for its inference speed, since a report that takes thirty seconds to generate feels very different from one that takes three.

7.6 Dashboard, Analytics, and PDF Export

On the frontend, all of the above is surfaced through a React dashboard: match score, recommended paths, strengths, missing skills, and the AI-generated narrative. A separate analytics view uses Recharts to plot skill distribution and career match visualizations. Finally, a one-click export (built with jsPDF) turns the whole report into a shareable, recruiter-friendly PDF document.

8. Technology Stack

Frontend

Technology	Purpose
React.js	Building the interactive user interface
React Router DOM	Client-side routing
Tailwind CSS	Responsive styling
Recharts	Analytics and data visualization
jsPDF	Client-side PDF report generation
EmailJS	Contact form integration

Backend

Technology	Purpose
FastAPI	REST API development
Python 3.11	Core backend language
Uvicorn	ASGI server for FastAPI

AI / Machine Learning

Technology	Purpose

Sentence Transformers	Semantic embeddings for resumes and job descriptions
Cosine Similarity	Core job-matching computation
Groq API (Llama 3.3 70B)	AI-generated career report
NumPy / Pandas	Numerical computation and dataset handling

Resume Processing

Technology	Purpose
PyMuPDF (fitz)	PDF text extraction
python-docx	DOCX resume parsing
Regular Expressions	Text cleaning and preprocessing

9. Project Structure

The codebase is organized to keep the frontend, backend, ML pipeline, data, and tests clearly separated rather than mixed together, which made it a lot easier to work on one part without accidentally breaking another:

- **frontend/** — React + Vite application: pages, components, layouts, and API calls.
- **backend/** — FastAPI routes, request/response schemas, and service logic.
- **src/** — Core Python package: resume parsing, embeddings, matching, skill-gap analysis, and the LLM report layer.
- **data/** — Raw and processed job description datasets used for matching.
- **models/** — Pre-computed job embeddings and the FAISS index used by the matching engine.
- **notebooks/** — Jupyter notebooks used for initial dataset exploration.
- **tests/** — Unit and integration tests covering the core pipeline modules.
- **docs/** — Supporting documentation, including dataset analysis notes.

10. Results and Discussion

The finished application covers the full flow end to end: a candidate lands on the landing page, uploads a resume through a drag-and-drop interface, and within a short wait is shown a dashboard with their overall match score, recommended career paths, identified strengths, and missing skills, alongside the AI-written narrative report. A separate analytics tab breaks the same information down visually — skill distribution charts, career-match comparisons, and a missing-skills overview — and the whole report can be exported as a formatted PDF in one click.

Qualitatively, the semantic matching does what it was meant to do: resumes that describe skills using different wording than the job postings still surface reasonable matches, which would not happen with pure keyword search. That said, I want to be honest about scope here — this observation is based on manual spot-checks during development rather than a formal, quantified evaluation across a labeled test set. Building out that kind of rigorous precision/recall evaluation is one of the concrete next steps for this

project, and is called out explicitly in the limitations section below.

11. Testing

The repository includes a dedicated **tests/** directory with unit tests covering the core pipeline modules — resume parsing, skill extraction, and the matching logic. These tests were mainly used during development to catch regressions while iterating on the pipeline, rather than as a full end-to-end test suite. Expanding this into a proper CI-integrated test suite, with automated runs on every change, is one of the near-term improvements planned for the project.

12. Challenges Faced

A few things were harder than expected while building this. Getting clean text out of PDFs turned out to be more finicky than I assumed going in — resumes come in wildly inconsistent layouts (multi-column designs, tables, embedded graphics), and PyMuPDF's default extraction sometimes scrambled the reading order on more design-heavy templates. Handling these edge cases without over-engineering the parser took a fair amount of trial and error.

On the matching side, tuning what counts as a "good" similarity score was less obvious than expected. Cosine similarity gives a continuous number, but translating that into a meaningful match score that feels intuitive to a non-technical user required some manual calibration rather than a clean formula.

Getting the Groq LLM output to stay consistently structured — so the frontend could reliably parse it into sections like strengths, gaps, and roadmap — also took a few iterations of prompt refinement. Early versions of the prompt occasionally produced report sections that didn't map cleanly onto the dashboard's expected fields.

13. Limitations and Future Scope

In the interest of giving an honest account of where the project currently stands, a few limitations are worth calling out directly:

- No formal, quantified evaluation of matching quality (precision/recall on a labeled test set) has been conducted yet — current confidence in the matching quality is based on manual review rather than metrics.
- Test coverage exists for core modules but is not yet comprehensive or wired into a continuous integration pipeline.
- Edge cases such as scanned or image-only resumes, non-English resumes, and malformed files are not fully handled yet.
- The job dataset is a static snapshot and does not reflect real-time market listings.

Planned future improvements include:

- A quantified evaluation pipeline with a labeled resume-job test set.
- Expanded automated testing with CI integration.
- ATS-style resume scoring and AI-generated cover letter drafting.
- User authentication with saved report history.
- Salary prediction and regional job-market analytics.

- A recruiter-facing dashboard for bulk resume analysis.

14. Conclusion

CareerCompass AI set out to answer a fairly specific question: can semantic similarity, combined with an LLM-generated narrative, produce a more useful and more honest picture of resume-to-job fit than traditional keyword matching? Building the full pipeline — parsing, embeddings, matching, LLM reporting, and a dashboard to tie it together — showed that the approach is workable end to end, and the qualitative results during development were encouraging.

At the same time, this project is a solid first version rather than a finished product. The honest gaps — no formal evaluation metrics yet, incomplete test coverage, and unhandled edge cases — are exactly the kind of things I'd want to close before calling this production-ready, and they form a clear roadmap for what comes next. Overall, the project achieved its core goal of demonstrating a working, semantically grounded career guidance pipeline, while leaving well-defined room to grow.

15. References and Acknowledgements

Core technologies and libraries used:

- Sentence Transformers, Hugging Face — semantic embeddings
- Groq API, Llama 3.3 70B — AI-generated career report
- FastAPI, Uvicorn — backend REST API
- React, Tailwind CSS, Recharts, jsPDF, Lucide React — frontend
- PyMuPDF, python-docx — resume parsing
- NumPy, Pandas — data processing
- EmailJS — contact form integration

Dataset:

lang-uk/recruitment-dataset-job-descriptions-english, Hugging Face Datasets (MIT License).

Thanks to the open-source community whose tools made this project possible, and to the mentors and peers in the internship program whose feedback shaped several of the design decisions along the way.